

**MBA
USP
ESALQ**

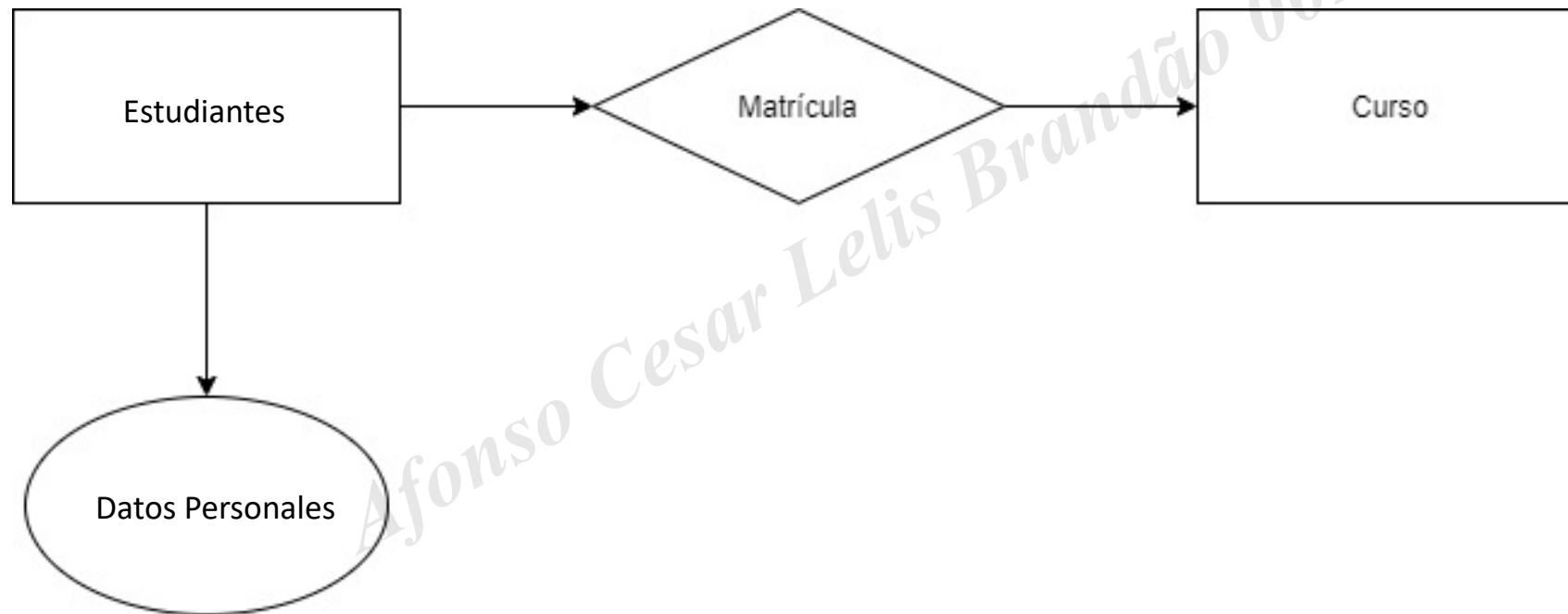
**INGENIERÍA DE
DATOS II**

Prof. Dr. Jeronymo Marcondes

***El profesor es responsable de la idoneidad, originalidad y legalidad de los contenidos didácticos presentados.**

Prohibida la reproducción total o parcial, sin autorización. Ley nº 9610/98

Introducción



Perguntas

- ¿Cómo unir las informaciones de las tablas?
- ¿Cómo obtener informaciones de curso a partir de los estudiantes?
- ¿Cómo realizar consultas de buen rendimiento calificaciones insertadas?
- ¿Cómo garantizar la solidez del modelo frente a transacciones?

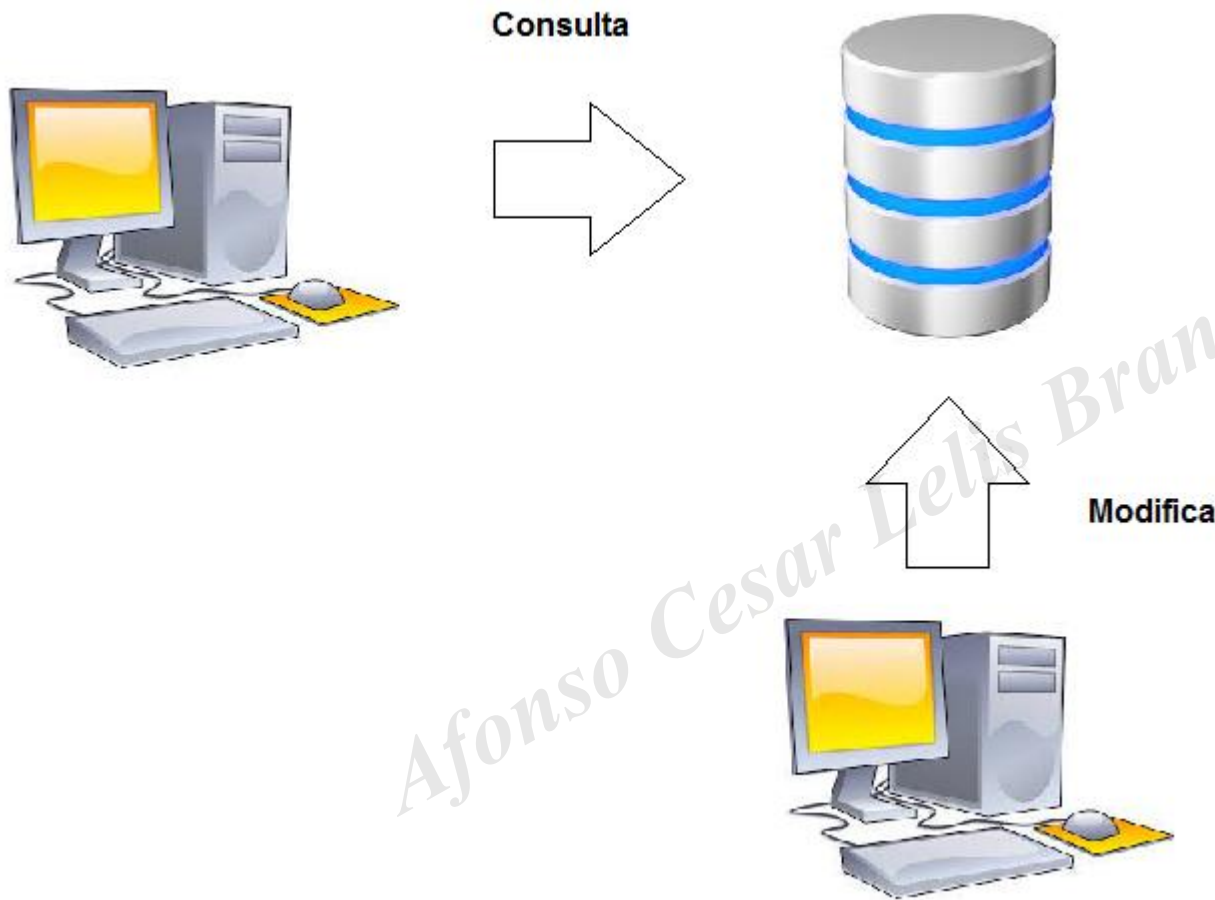
Transacciones

- Ejecución en el banco de datos.
- Garantiza integridad – ejemplo de la compra de butacas de teatro.
- Sensación de ejecución local con aislamiento y protección contra las pérdidas.
- Concepto de *lock*.

Locking Protocol

- Reglas que garantizan que, aunque varias personas ejecuten *queries* al mismo tiempo, el resultado neto será el mismo que si se hubieran ejecutado en la fila.
- El *lock* garantizará que el objeto consultado no pueda ser accedido por medio de otras transacciones.
- *Exclusive Lock* y *Shared Lock*.

Ejemplo



Relación

- ¿Cómo garantizar solidez al modelo del negocio? Además de la solidez de las transacciones -> restricciones.
- La relación entre entidades. ¿Cómo se relacionan los estudiantes y los cursos?
- La tabla debe tener consistencia interna y también sus relaciones con otras tablas.

Restricciones de Integridad

- Restricciones de clave, de relación y generales.
- Restricciones de clave: un subconjunto mínimo de los campos de una relación que identifica una única tupla.
- Es decir, los campo(s) definidos como clave deben garantizar que la línea seleccionada sea única.

Ejemplo

CPF	Nombre	Curso
xxx	João	Ciencia de Datos
yyy	João	Medicina
hhh	Pedro	Medicina

Nombre	Apellido	Curso
João	Silva	Ciencia de Datos
João	Marinho	Ciencia de Datos
Pedro	Guedes	Ciencia de Datos

Clave Primaria

- Una determinada tabla puede tener varias claves = claves candidatas
- Clave primaria es definida por el DBA de forma que el SGBD haga las investigaciones por medio de la misma.
- La clave primaria bien definida es importante porque suscita la creación y los índices, lo que hace que las consultas sean más eficaces.

Formas Normales

- Serie de reglas que garantizan el buen diseño de una BD.
- Muestra la importancia de una clave primaria bien definida.
- Objetivo:
 - 1) Garantizar la información sin redundancia.
 - 2) Garantizar la eficiencia en la obtención de los datos.

Formas normales

- 1ª forma normal:

Cada línea es una información. No pueden existir los grupos repetidos o atributos con más de un valor.

PERSONAS = { ID + NOMBRE + DIRECCION + TELEFONOS }

PERSONAS = { ID + NOMBRE + DIRECCION }

TELEFONOS = { PERSONA ID + TELEFONO }

Formas normais

- 1ª forma normal:

ID	NOMBRE	DIRECCIÓN	TELEFONOS
XX	JOAO	AV JOAO	99999;88888;77777
YY	PEDRO	AV PEDRO	77776;5555

ID	NOMBRE	DIRECCIÓN
XX	JOAO	AV JOAO
YY	PEDRO	AV PEDRO

ID	TELEFONO
XX	99999
XX	88888
XX	77777
YY	77776
YY	5555

Formas normales

- 2ª forma normal:

Todas las columnas que no participan de la clave primaria son dependientes de todas las columnas que componen la clave primaria.

ALUMNOS_CURSOS = { ID ALUMNO + ID CURSO + NOTA + DESCRIPCION_CURSO }

ALUMNOS_CURSOS = { ID ALUMNO + ID CURSO + NOTA }

CURSOS = { ID CURSO + DESCRIPCION }

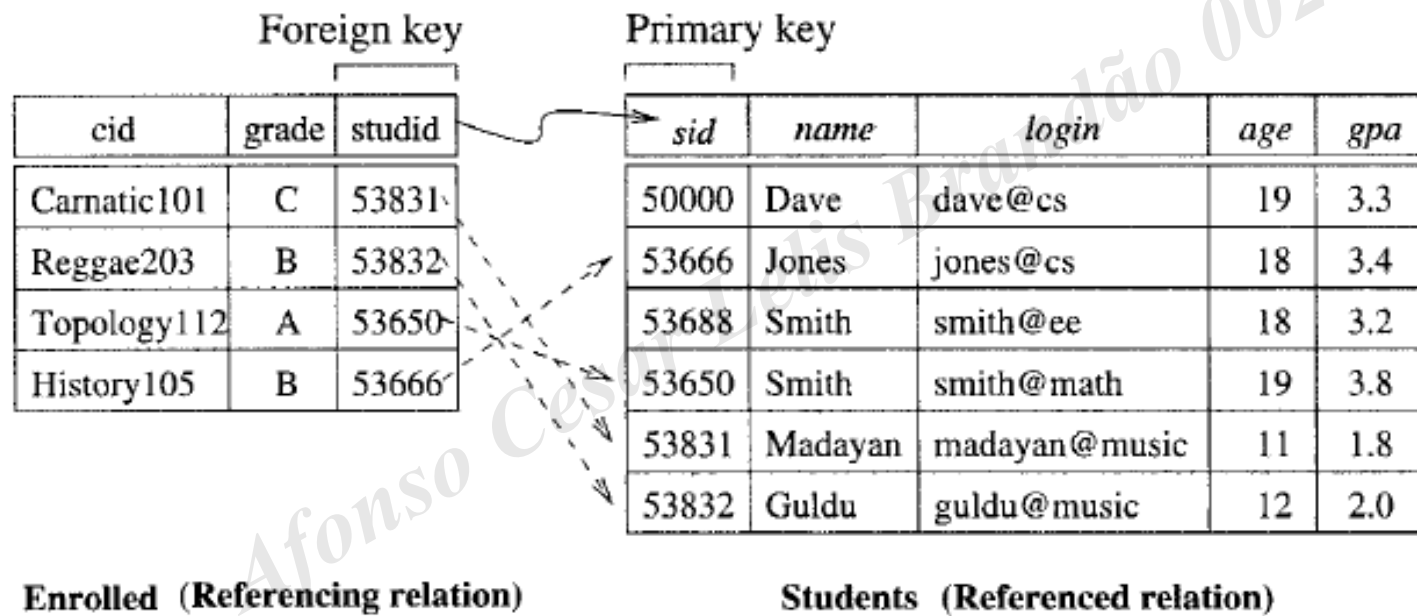
Restricciones Generales

- Restricciones, principalmente, de negocio.
- Ejemplo: inserción de edad.
- Los modernos SGBD ya tienen herramientas que permiten crear estas restricciones.

¿Cómo lidar con los modelos con más de una
tabla?

Clave Extranjeras

- Clave primaria de otra tabla
- Esta clave nos permite vincular diferentes tablas para garantizar la unicidad de la relación.
- El nombre de la clave extranjera no necesita ser el mismo de la clave primaria = ¡lo que importa es el contenido!



Casos Específicos

- Inserir $\langle 55555, \text{Art104}, A \rangle$ en los cursos con inscrição.
- Borrar tupla $\langle 53666, \text{Jones}, \text{Jones}, \text{Jones}@cs, 18, 3.4 \rangle$ de estudantes.
- Inserir tupla $\langle 55669, \text{Margareth}, \text{MG}@test, 21, 4 \rangle$ en estudantes.

Idea de unión

Tabla 1

CPF	NOMBRE
xxx	ze das couves
yyy	maria das desgraças

Tabla 2

CPF	EDAD	PIS
xxx	21	hhh
yyy	25	JJJ

Tabla Derivada

CPF	NOMBRE	EDAD
xxx	ze das couves	21
yyy	maria das desgraças	25

ACID

- *Atomicity, Consistency, Isolation, Durability*
- Conjunto de las propiedades en las transacciones de los bancos de datos que son importantes para garantizar la validez de los datos incluso si ocurren los errores durante el almacenamiento o problemas más graves en el sistema, como los *crashes* o problemas físicos en un servidor. Las propiedades *ACID* son fundamentales para el procesamiento de las transacciones en los bancos de datos.

ACID

- Atomicidad: Garantiza que cada transacción sea tratada como una única entidad, la que debe ser ejecutada por completo o fallar completamente.
- Consistencia: Los datos que son grabados deben ser válidos.
- Aislamiento: Permite dejar el banco de datos en el mismo estado en que estaría si las transacciones fueran ejecutadas en secuencia.
- Durabilidad: La propiedad de la durabilidad garantiza que una transacción, una vez ejecutada (efectiva), permanecerá en este estado, aunque haya un problema grave en el sistema.

Cardinalidad

- Cardinalidad: indica cuántas ocurrencias de una Entidad participan en el mínimo y el máximo de la relación.

Tipos de relación:

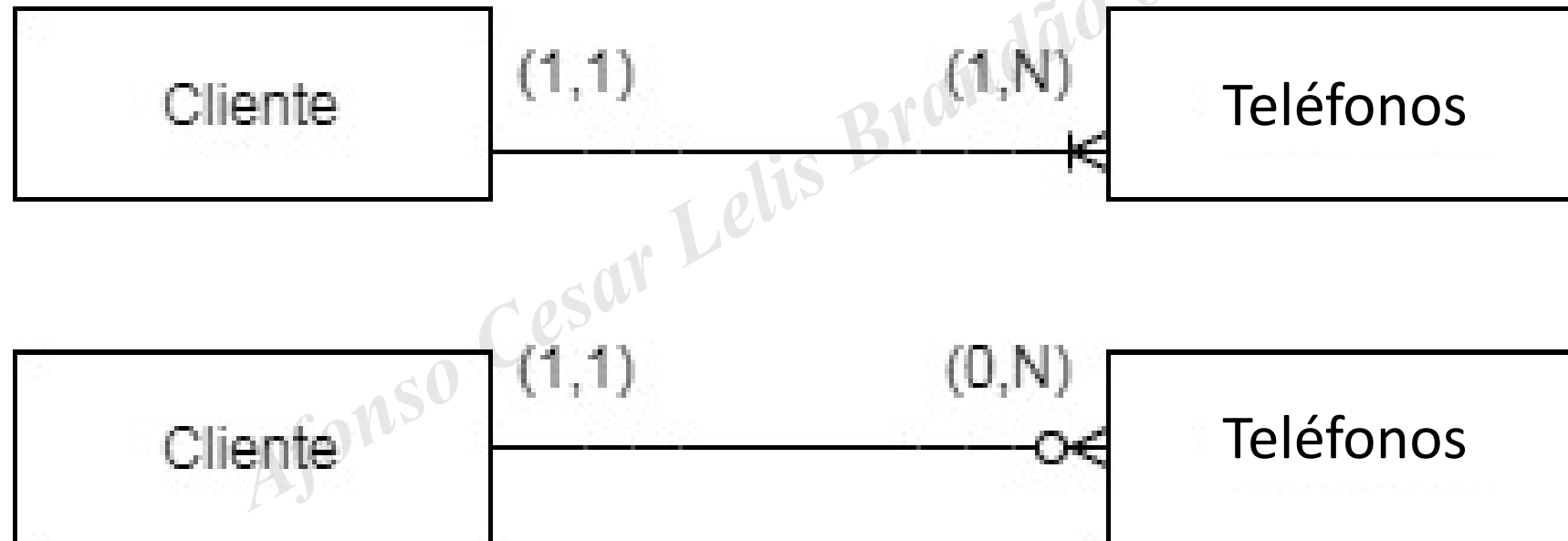
- 1) Uno para uno;
- 2) Muchos para uno;
- 3) Muchos para muchos.

Uno para uno

- Cardinalidade mínima: define se a relação é obrigatória.
- Cardinalidade máxima: define a quantidade máxima de ocorrências de a Entidade que podem participar na Relação.



Muchos para uno



Muchos para Muitos



Operando con SQL

JOIN

- Especifica cómo será realizada la unión entre dos tablas. Por ejemplo:

Id_cliente	Pedido

Id_cliente	Nombre	Dirección

Id_cliente	Nombre	Dirección	Pedido

LEFT JOIN

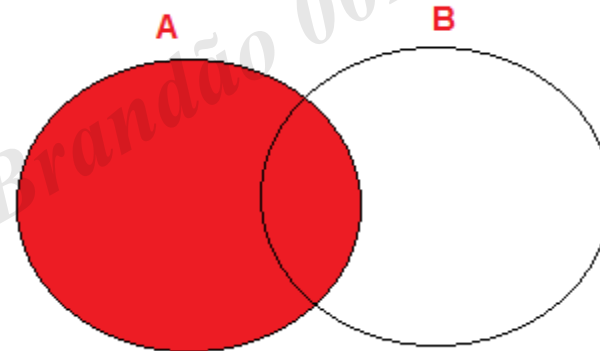
SELECT [DISTINCT] lista-selección

FROM lista-origen-1

LEFT JOIN lista-origen-2

ON lista-origen-1. campo_en_común = lista-origen-2. campo_en_común

WHERE calificación



LEFT JOIN

SELECT *

FROM pedidos

LEFT JOIN dirección

ON pedidos.Id_cliente = dirección.Id_cliente

Id_cliente	Pedido
xxx	1
yyy	2

Id_cliente	Nombre	Dirección
xxx	joao	av joao
hhh	pedro	av pedro

Id_cliente	Nombre	Dirección	Pedido
xxx	joao	av joao	1
yyy	NULL	NULL	2

NULL

- Hasta ahora solamente valores conocidos.
- Si es desconocido = *NULL*.
- Cuando el valor es desconocido o no se aplica.

Ejemplo con *NULL*

SELECT *
FROM pedidos_y_dirección
WHERE nombre *IS NOT NULL*

Id_cliente	Nombre	Dirección	Pedido
xxx	joao	av joao	1

SELECT *
FROM pedidos_y_dirección
WHERE nombre *IS NULL*

Id_cliente	Nombre	Dirección	Pedido
yyy	NULL	NULL	2

RIGHT JOIN

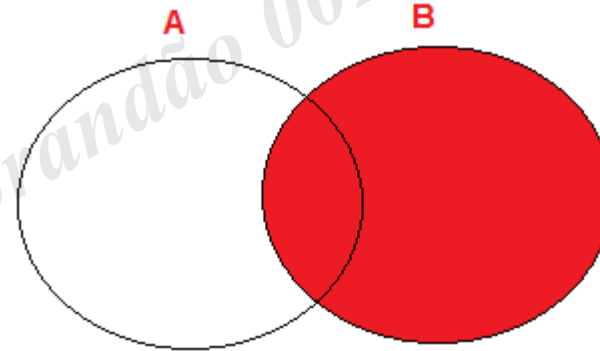
SELECT [*DISTINCT*] lista-selección

FROM lista-origen-1

RIGHT JOIN lista-origen-2

ON lista-origen-1. campo_en_común = lista-origen-2. campo_en_común

WHERE calificación



RIGHT JOIN

SELECT *

FROM pedidos

RIGHT JOIN dirección

ON pedidos.Id_cliente = dirección.Id_cliente

Id_cliente	Pedido
xxx	1
yyy	2

Id_cliente	Nombre	Dirección
xxx	joao	av joao
hhh	pedro	av pedro

Id_cliente	Nombre	Dirección	Pedido
xxx	joao	av joao	1
hhh	pedro	av pedro	NULL

INNER JOIN

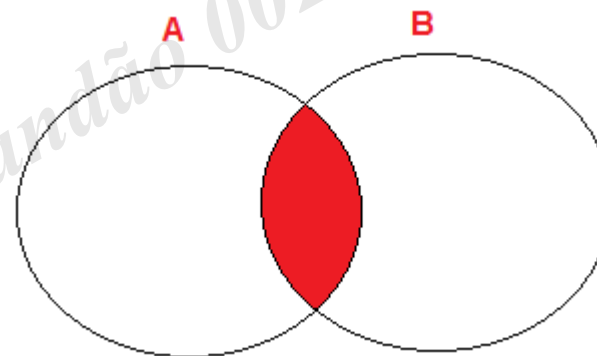
SELECT [*DISTINCT*] lista-selección

FROM lista-origen-1

INNER JOIN lista-origen-2

ON lista-origen-1. campo_en_común = lista-origen-2. campo_en_común

WHERE calificación



INNER JOIN

SELECT *

FROM pedidos

INNER JOIN dirección

ON pedidos.Id_cliente = dirección.Id_cliente

Id_cliente	Pedido
xxx	1
yyy	2

Id_cliente	Nombre	Dirección
xxx	joao	av joao
hhh	pedro	av pedro

Id_cliente	Nombre	Dirección	Pedido
xxx	joao	av joao	1

UNION ALL

- Tablas con la misma estructura que serán “apiladas”.

SELECT lista-selección

FROM lista-origen-1

UNION ALL

SELECT lista-selección

FROM lista-origen-2

UNION ALL

SELECT **Id_cliente**

FROM **pedidos**

UNION ALL

SELECT **Id_cliente**

FROM **dirección**

Id_cliente	Pedido
xxx	1
yyy	2

Id_cliente	Nombre	Dirección
xxx	joao	av joao
hhh	pedro	av pedro

Id_cliente
xxx
yyy
xxx
hhh

UNION

- Tablas con la misma estructura que serán “apiladas”.
- Se diferencia por aplicar un *DISTINCT*.

SELECT lista-selección

FROM lista-origen-1

UNION

SELECT lista-selección

FROM lista-origen-2

UNION

SELECT **Id_cliente**

FROM **pedidos**

UNION

SELECT **Id_cliente**

FROM **dirección**

Id_cliente	Pedido
xxx	1
yyy	2

Id_cliente	Nombre	Dirección
xxx	joao	av joao
hhh	pedro	av pedro

Id_cliente
xxx
yyy
hhh

Nested Queries

- Resultado del *query* anterior puede ser utilizada en la actual
- Forma más común:

SELECT lista-selección-derivada

FROM lista-origen

WHERE columna **IN**

(

SELECT lista-selección-original

FROM lista-origen

)

Nested Queries

CPF	Nombre	Dirección
XXX	JOAO	AV JOAO
YYY	MARIA	AV MARIA

CPF	PEDIDO	VALOR (R\$)
XXX	10	500
YYY	12	1000

Nested Queries

```
SELECT CPF, Nombre, Dirección  
FROM Consumidores  
WHERE CPF IN  
(  
    SELECT CPF  
    FROM Gastos  
    WHERE Valor > 500  
)
```

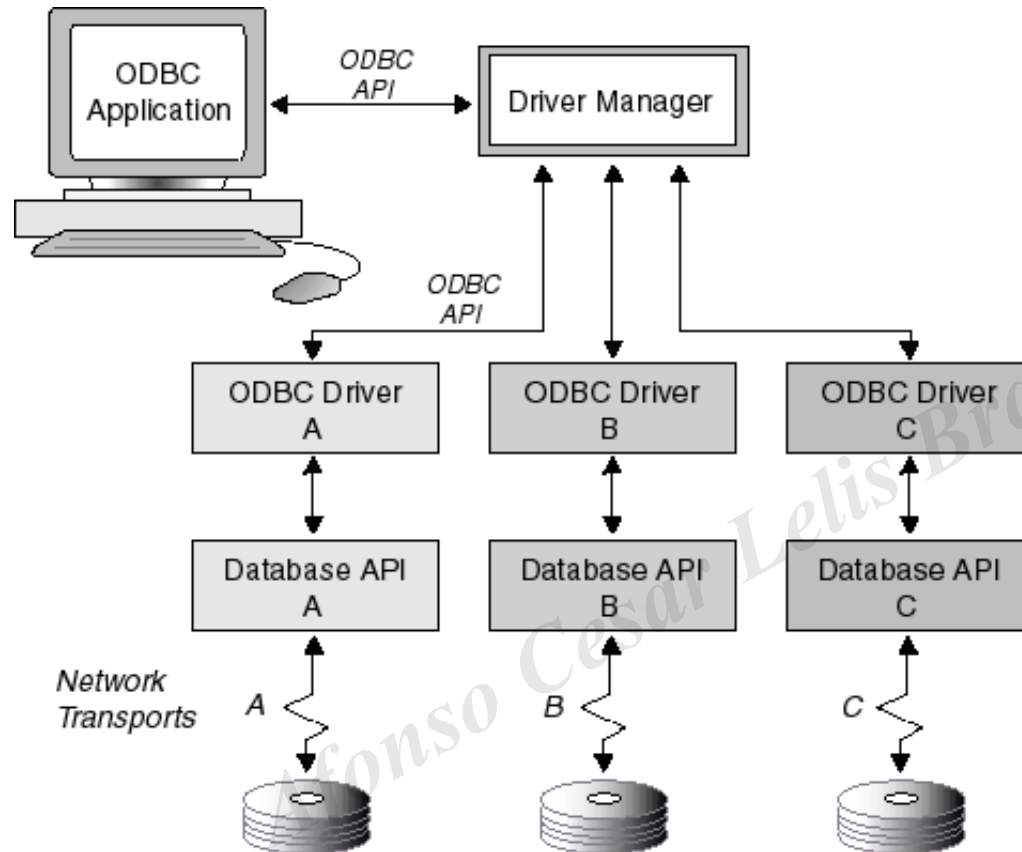
ODBC y JDBC

- *Java Database Connectivity – SUM*
- *Open Database Connectivity – Microsoft*
- *API – application programming interface*

ODBC y JDBC

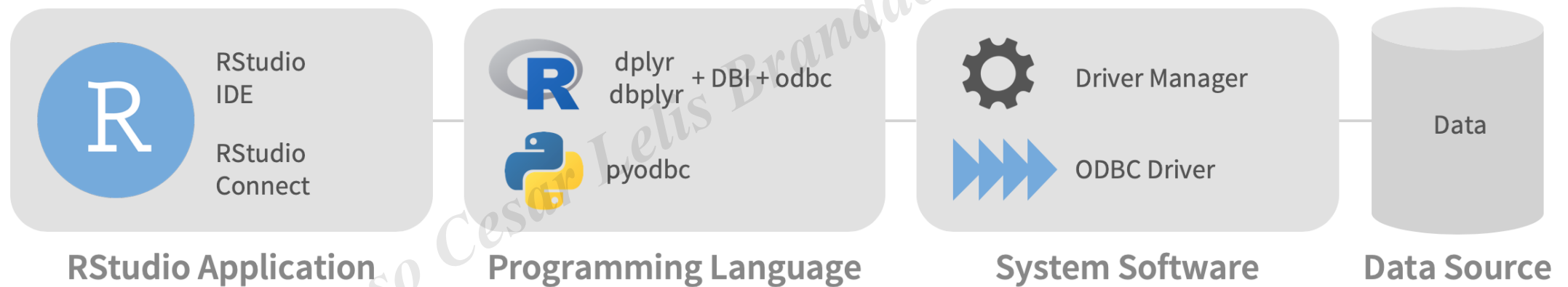
- Permite la ejecución de SQL dentro del banco a partir de las aplicaciones.
- Puede acceder los varios servidores de datos al mismo tiempo.
- Todas las transacciones ocurren por medio de un **driver**.

ODBC y JDBC



<https://docs.oracle.com/>

Ejemplo en el *R Studio*



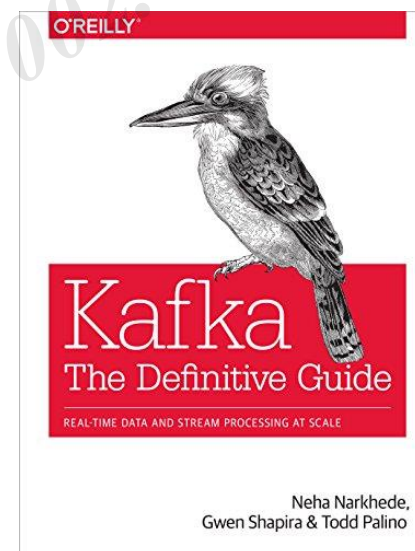
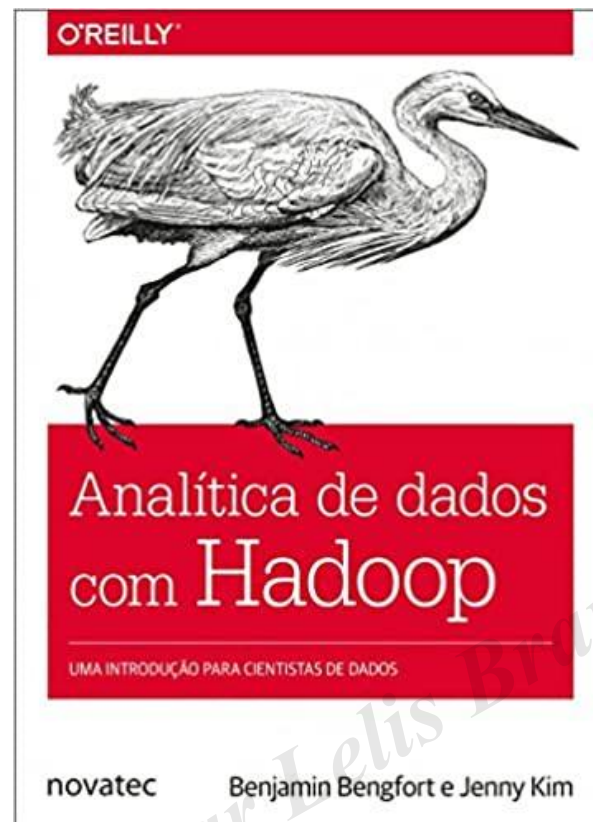
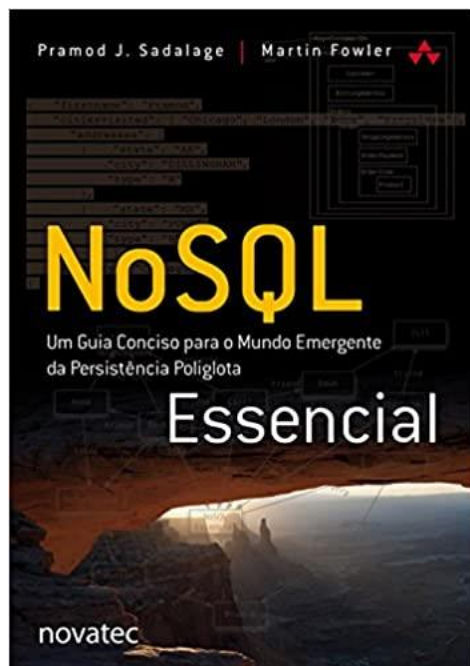
ODBC y JDBC

- Orden:
 1. Selecciona el origen de los datos.
 2. Carga el respectivo *driver*.
 3. Establece la conexión con el origen.

ODBC y JDBC

- Cada conexión tiene sus características.
- Se define la cadena de conexión con el banco.

```
con <- DBI::dbConnect(odbc::odbc(),  
                      Driver = "[your driver's name]",  
                      Server  = "[your server's path]",  
                      UID      = rstudioapi::askForPassword("Database user"),  
                      PWD      = rstudioapi::askForPassword("Database password"),  
                      Port     = 3306)
```



Discussão – el futuro de los bancos de datos

MBA
USP
ESALQ

Prof. Dr. Jeronymo Marcondes



<https://www.linkedin.com/in/jeronymo-marcondes-585a26186>